IoT Challenge 3 – Node-RED

Rong Huang – 10948935

May 9, 2026

1 Introduction

This report describes the Node-RED flow developed for IoT Challenge 3. The flow periodically generates MQTT messages, processes the provided `challenge3.csv` file, extracts ZigBee measurement and link-status information, writes the required CSV files, and sends selected outgoing cost values to ThingSpeak.

2 Execution Limit

The flow is configured to process exactly 200 ID messages received from the MQTT subscription. The limit is implemented using a flow-level counter, `id_counter`, inside the ID generation function. For each generated message, the counter is increased by one. When the counter reaches 200, the function returns `null`, so no further MQTT message is published and no additional CSV row is written.

Messages that do not contain either a `ZBEE_ZCL` layer or a `Link Status (0x08)` packet are ignored by the processing branches. However, they still count toward the 200-message limit because the limit is applied at the ID-message level, before checking the packet type.

3 ThingSpeak

A ThingSpeak channel was configured to receive the selected outgoing cost values. Only `Field 1` was used, and it was named `Outgoing Cost`. The channel was made public as required by the assignment.

In the Node-RED flow, the `Build URL` function creates an HTTP GET request using the ThingSpeak Write API key. For each selected destination address, the corresponding outgoing cost is sent to `field1`. The `ThingSpeak 20s delay` node ensures that the HTTP requests are sent at a rate of one message every 20 seconds.

ThingSpeak Channel ID: 3370197

ThingSpeak Channel Link: `https://thingspeak.mathworks.com/channels/3370197`

4 Node-RED Flow

Figure 1 shows the complete Node-RED flow. The flow is divided into the ID generation branch, packet selection branch, ZBEE_ZCL processing branch, Link Status processing branch, and ThingSpeak publishing branch.

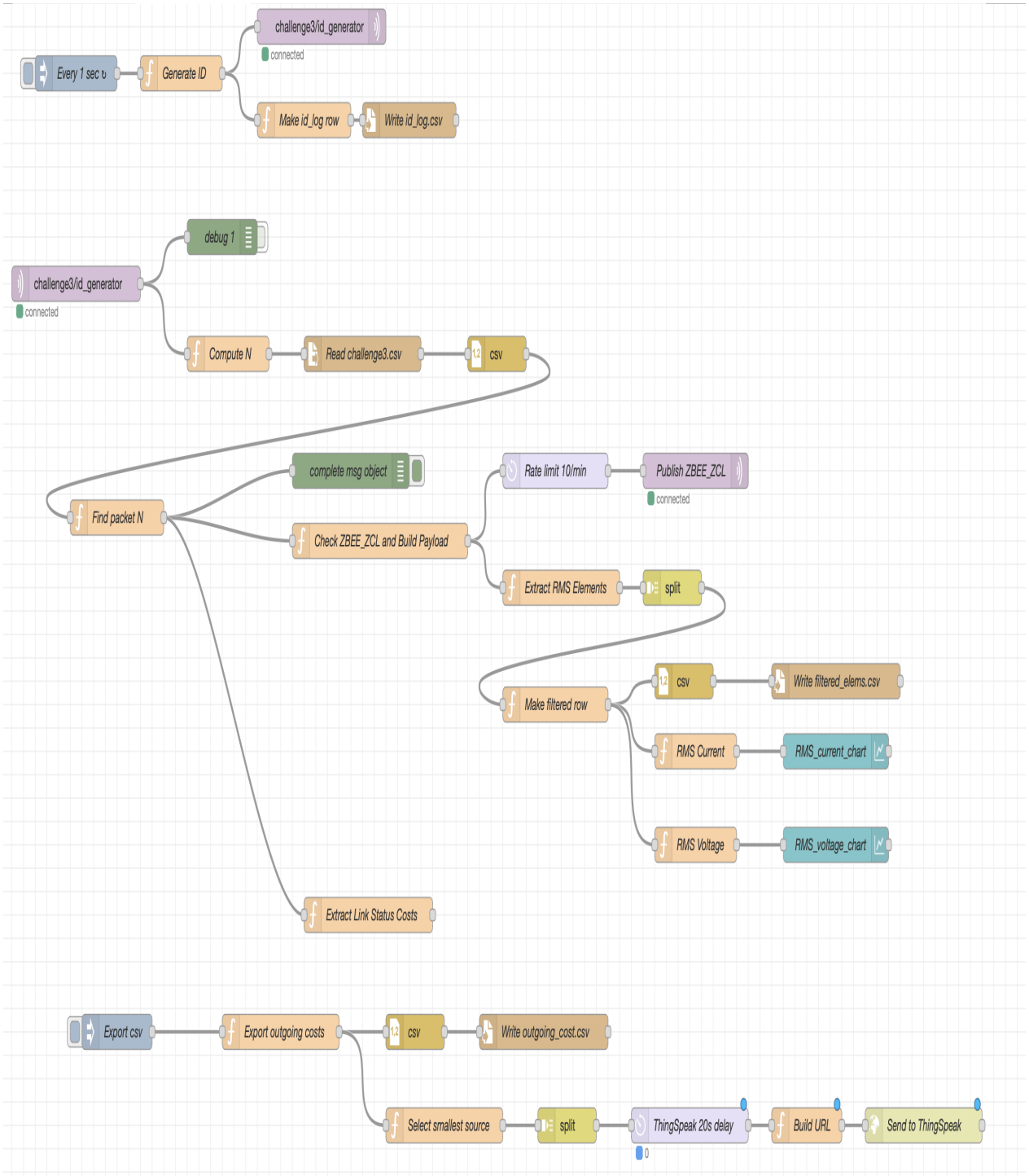


Figure 1: Complete Node-RED flow.

5 Challenge Implementation

5.1 Question 1: ID Generation and MQTT Publishing

This part is implemented using the following nodes:

- `Inject`: triggers the flow every second.
- `Generate ID`: generates a random ID between 0 and 30000 with the current UNIX timestamp.
- `MQTT out`: publishes the JSON message to `challenge3/id_generator`.
- `Make id_log row`: converts the generated fields into CSV format.
- `Write file`: appends the row to `id_log.csv`.

The first branch generates one MQTT message every second. Each message contains a random ID and a UNIX timestamp. The same information is also saved in `id_log.csv` using the required format:

```
No.,ID,TIMESTAMP
```

The sample output of `id_log.csv` is shown in Figure 2.

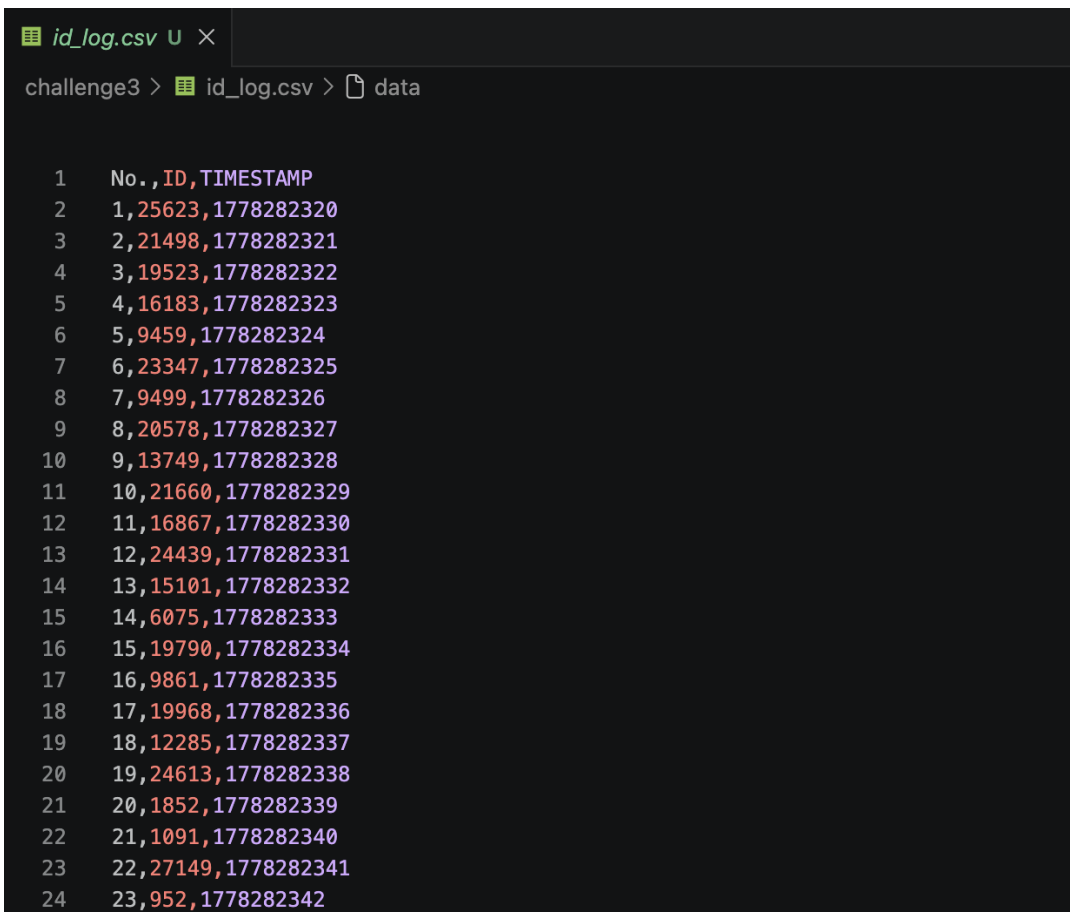


Figure 2: Sample output of `id_log.csv`.

5.2 Question 2: Packet Processing and Measurement Extraction

This part is implemented using the following nodes:

- `MQTT in`: subscribes to `challenge3/id_generator`.
- `Compute N`: computes $N = ID \bmod 5218$.
- `Read file`: reads `challenge3.csv`.

- `CSV`: parses the file into structured objects.
- `Find packet N`: selects the row with `Packet Number = N`.
- `Check ZBEE_ZCL and Build Payload`: checks ZBEE_ZCL packets and builds the required JSON payload.
- `Rate limit 10/min`: limits MQTT publishing to 10 messages per minute.
- `MQTT out`: publishes the processed ZBEE_ZCL message.
- `Extract RMS Elements`: extracts Active Power, RMS Current, and RMS Voltage data.
- `Split`: sends each extracted element as a separate message.
- `Make filtered row`: formats each extracted element for CSV output.
- `CSV` and `Write file`: generate `filtered_elems.csv`.
- `Chart nodes`: visualize RMS Current and RMS Voltage.

For each received ID, the flow computes:

$$N = ID \bmod 5218$$

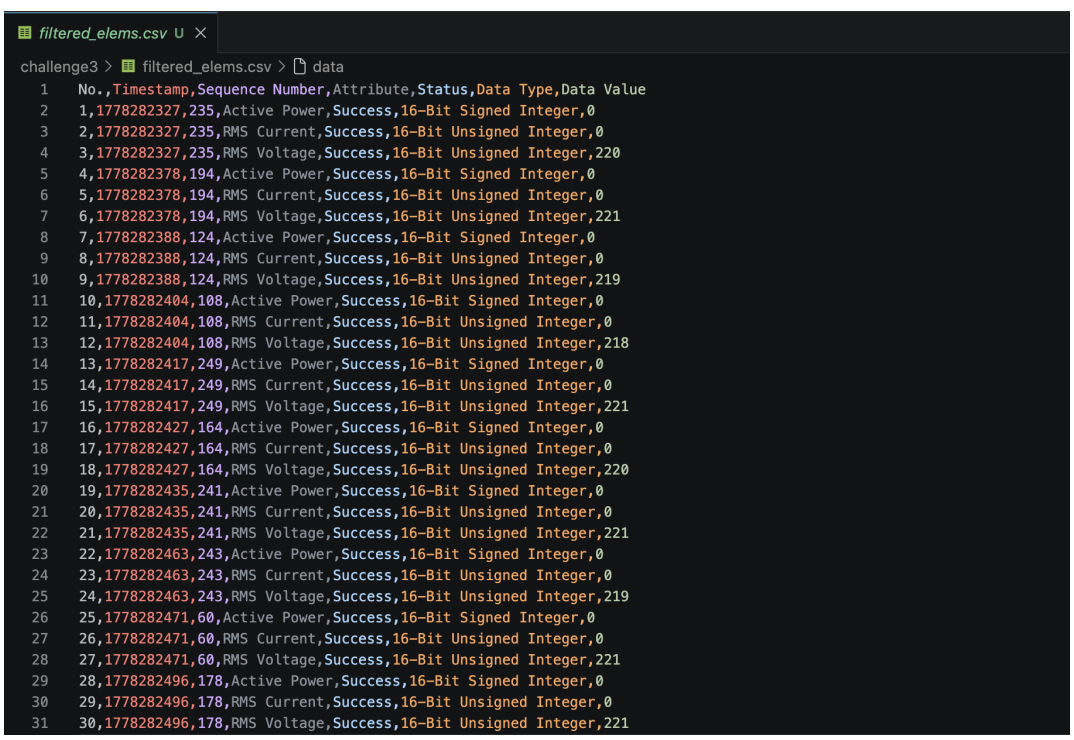
Then it searches `challenge3.csv` for the row whose packet number is equal to N.

If the selected row contains a `ZBEE_ZCL` layer, the flow builds a new JSON payload and publishes it to the topic based on the ZigBee source device name. The publication is rate-limited to 10 messages per minute.

If the command string contains `Active Power`, `RMS Current`, or `RMS Voltage`, the flow extracts the corresponding attribute, status, data type, and value. These fields are matched by their positions in the corresponding lists. The extracted rows are saved in `filtered_elems.csv` using the required format:

```
No.,Timestamp,Sequence Number,Attribute,Status,Data Type,Data Value
```

The sample output of `filtered_elems.csv` is shown in Figure 3.




```
No.,Timestamp,Sequence Number,Attribute,Status,Data Type,Data Value
1,1778282327,235,Active Power,Success,16-Bit Signed Integer,0
2,1778282327,235,RMS Current,Success,16-Bit Unsigned Integer,0
3,1778282327,235,RMS Voltage,Success,16-Bit Unsigned Integer,220
4,1778282378,194,Active Power,Success,16-Bit Signed Integer,0
5,1778282378,194,RMS Current,Success,16-Bit Unsigned Integer,0
6,1778282378,194,RMS Voltage,Success,16-Bit Unsigned Integer,221
7,1778282388,124,Active Power,Success,16-Bit Signed Integer,0
8,1778282388,124,RMS Current,Success,16-Bit Unsigned Integer,0
9,1778282388,124,RMS Voltage,Success,16-Bit Unsigned Integer,219
10,1778282404,108,Active Power,Success,16-Bit Signed Integer,0
11,1778282404,108,RMS Current,Success,16-Bit Unsigned Integer,0
12,1778282404,108,RMS Voltage,Success,16-Bit Unsigned Integer,218
13,1778282417,249,Active Power,Success,16-Bit Signed Integer,0
14,1778282417,249,RMS Current,Success,16-Bit Unsigned Integer,0
15,1778282417,249,RMS Voltage,Success,16-Bit Unsigned Integer,221
16,1778282427,164,Active Power,Success,16-Bit Signed Integer,0
17,1778282427,164,RMS Current,Success,16-Bit Unsigned Integer,0
18,1778282427,164,RMS Voltage,Success,16-Bit Unsigned Integer,220
19,1778282435,241,Active Power,Success,16-Bit Signed Integer,0
20,1778282435,241,RMS Current,Success,16-Bit Unsigned Integer,0
21,1778282435,241,RMS Voltage,Success,16-Bit Unsigned Integer,221
22,1778282463,243,Active Power,Success,16-Bit Signed Integer,0
23,1778282463,243,RMS Current,Success,16-Bit Unsigned Integer,0
24,1778282463,243,RMS Voltage,Success,16-Bit Unsigned Integer,219
25,1778282471,60,Active Power,Success,16-Bit Signed Integer,0
26,1778282471,60,RMS Current,Success,16-Bit Unsigned Integer,0
27,1778282471,60,RMS Voltage,Success,16-Bit Unsigned Integer,221
28,1778282496,178,Active Power,Success,16-Bit Signed Integer,0
29,1778282496,178,RMS Current,Success,16-Bit Unsigned Integer,0
30,1778282496,178,RMS Voltage,Success,16-Bit Unsigned Integer,221
```

Figure 3: Sample output of `filtered_elems.csv`.

The extracted RMS Voltage and RMS Current values are plotted using two Node-RED dashboard charts. The dashboard was accessed locally at `http://127.0.0.1:1880/ui`.

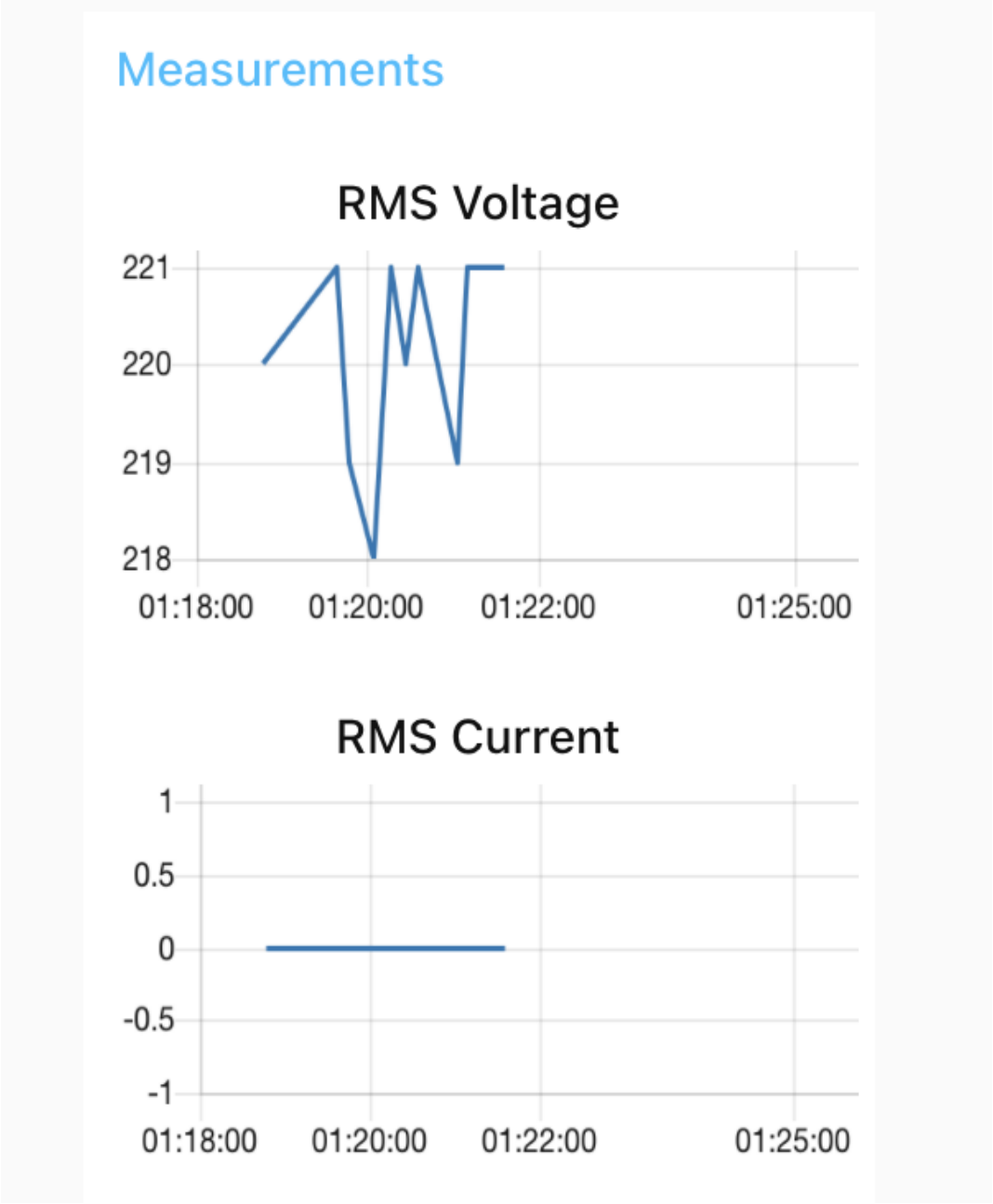


Figure 4: Node-RED dashboard charts for RMS Voltage and RMS Current.

5.3 Question 3: Link Status, Outgoing Cost, and ThingSpeak

This part is implemented using the following nodes:

- `Extract Link Status Costs`: detects Link Status packets and stores source-destination outgoing costs.
- `Export csv`: manually triggers the export after the 200-message execution.

- `Export outgoing costs`: converts the stored cost table into CSV rows.
- `CSV` and `Write file`: generate `outgoing_cost.csv`.
- `Select smallest source`: finds the smallest source address and sorts its destinations.
- `Split`: sends each destination-cost pair separately.
- `ThingSpeak 20s delay`: limits HTTP transmission to one message every 20 seconds.
- `Build URL`: builds the ThingSpeak HTTP GET request.
- `HTTP request`: sends the outgoing cost to ThingSpeak field 1.

If the selected packet is a `Link Status (0x08)` packet, the flow extracts the ZigBee source address and the outgoing cost for each destination link. The source-destination pair is used as the key, so if a later Link Status packet from the same source updates the same destination, the stored cost is updated instead of duplicated.

At the end of the execution, the stored link-cost table is exported to `outgoing_cost.csv` using the required format:

```
No.,Source,Destination,Cost
```

The sample output of `outgoing_cost.csv` is shown in Figure 5.

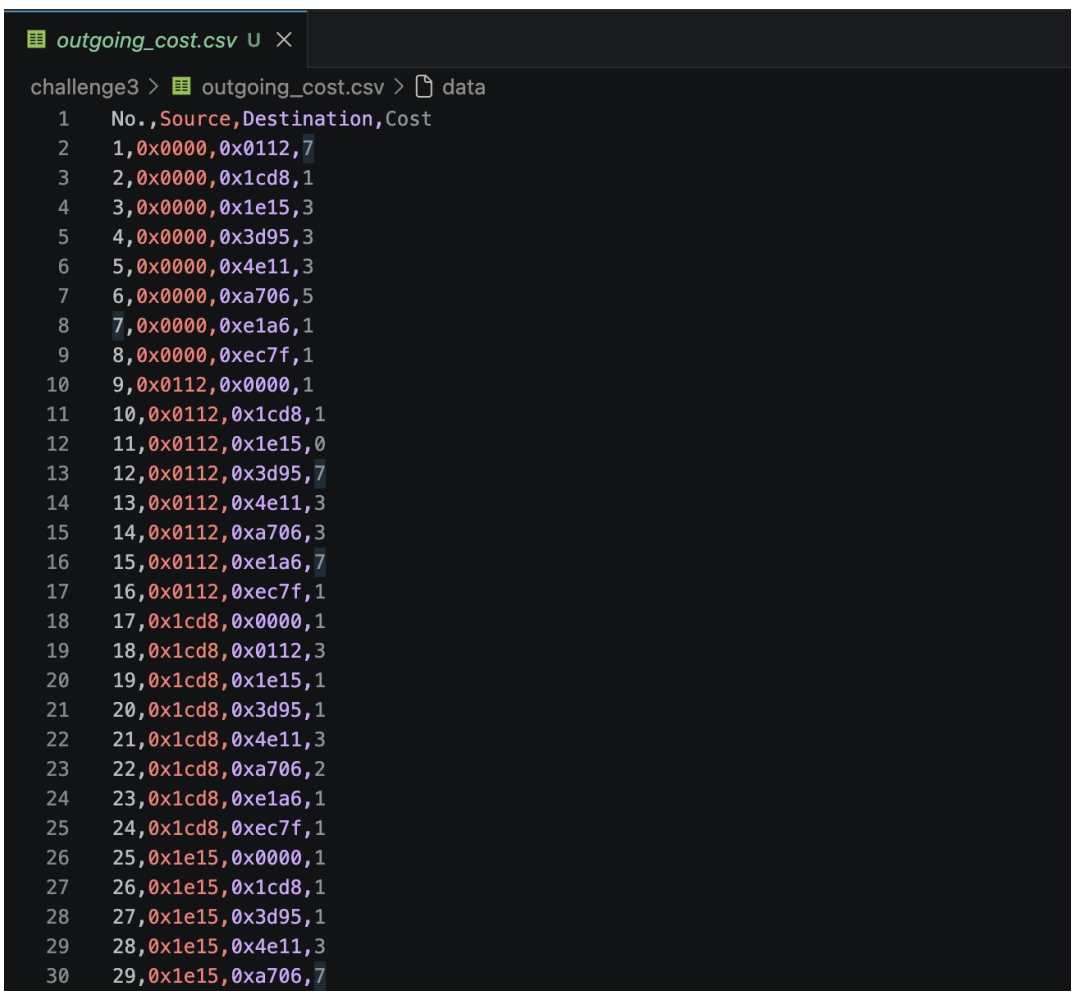


Figure 5: Sample output of `outgoing_cost.csv`.

The smallest source address is then selected by interpreting addresses as hexadecimal numbers. For this source, the destination addresses are sorted in ascending hexadecimal order. The corresponding outgoing costs are sent to ThingSpeak field 1 with a rate limit of one message every 20 seconds.

ThingSpeak Channel ID: 3370197

ThingSpeak Channel Link: `https://thingspeak.mathworks.com/channels/3370197`

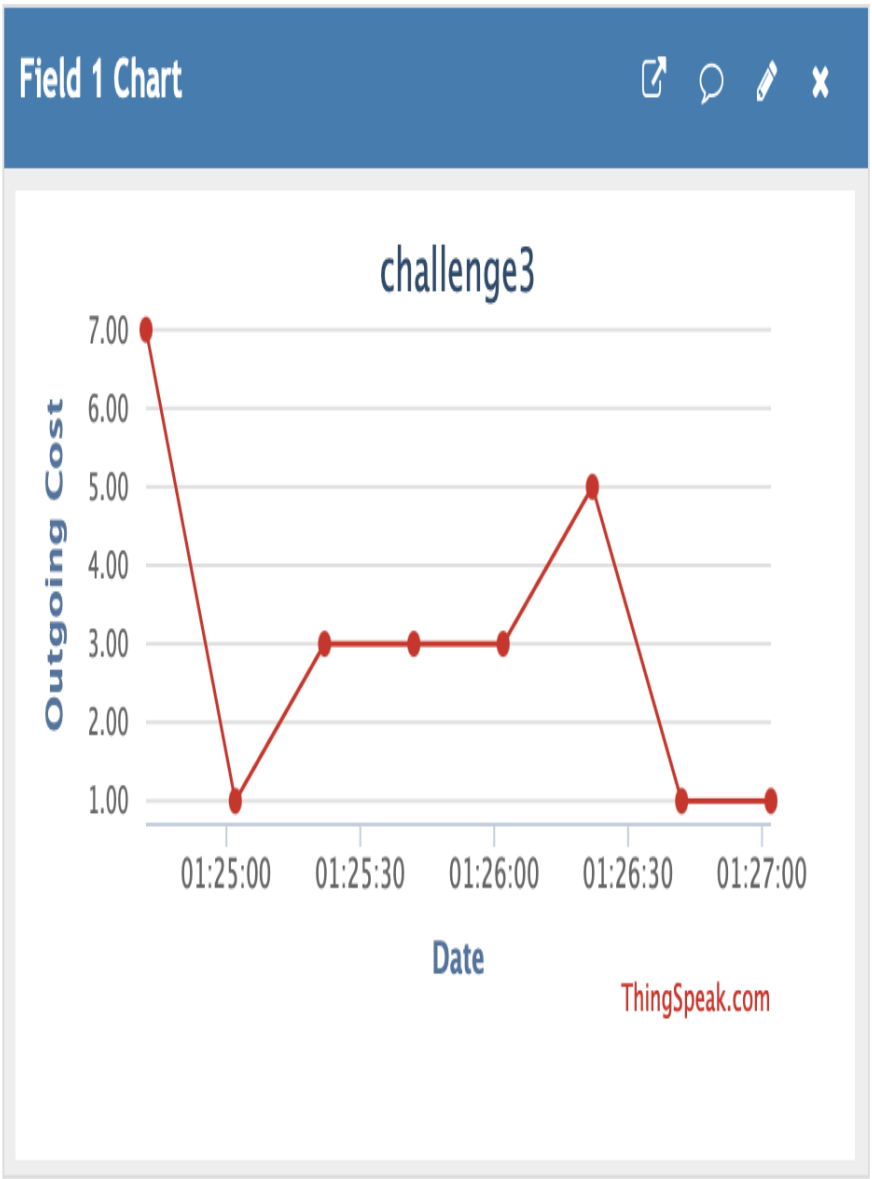


Figure 6: ThingSpeak chart showing outgoing cost values.